# 第3章 主存储器

# 及数据在计算机内的表示形式

| 厂商 | 赛道 | 代表产品 | 市场地位 |
| --- | --- | --- | --- |
| 长鑫存储 | DRAM | DDR5、LPDDR5X | 全球 8%，国内第一 |
| 长江存储 | 3D NAND | 294 层 NAND、致态 SSD | 全球 13%，国内第一 |
| 兆易创新 | NOR/DRAM | SPI NOR、利基 DRAM | NOR 全球第二 |
| 澜起科技 | 内存接口 | DDR5 接口芯片 | 全球 75% 市占 |
| 江波龙 | 存储模组 | Lexar SSD、内存条 | 国内模组龙头 |
| 佰维存储 | SSD / 嵌入式 | 工业级 SSD | 自研主控 + 颗粒 |

# 第3章主存储器及数据在计算机内的表示形式



3.1 主存储器

数据存储的基本形式、数据地址的类型及转换

3.2 数值数据在计算机内的表示形式

3.3 字符数据在机内的表示形式

3.4 数据类型转换

3.5 浮点数据在机内的表示形式

## 学习重点

数据存储的基本形式

有符号整数和无符号整数的表示与存储

字符串的表示方法与存储

浮点数据的表示方法与存储

地址类型转换 与 数据类型转换

# 课堂目标



- 给定一个整数，能正确给出其补码表示形式及存储结果
- 给定一个浮点数，能正确给出其表示形式及存储结果
- 给定一个字符串，能正确给出其表示形式及存储结果
- 给定一个结构数组，能正确给出其存储结果

对于 C 语句中定义的整型、浮点型、字符数组、结构数组、指针等能正确画出内存中存储的示意图；对于内存中看到的存储结果，能够写出对应的 C语言赋值语句。

对不同长度的整型变量之间的赋值，能够给出赋值结果。

## 3.1.1 数据存储的基本形式

内存条：用来存放程序和数据的装置

| | | | | | |
|---|---|---|---|---|---|
| C 语 言： | char、 | short、 | int、 | float、 | double |
| 对应长度： | 1个、 | 2个、 | 4个、 | 4个、 | 8个字节 |
| | 字节 | 字 | 双字 | 双字 | 四字 |
| 汇编语言： | byte | word | dword | dword | qword |

字节是最小的寻址单位

每一个字节都有一个地址

物理地址 (Physical Address, PA) 是唯一的

| | |
|---|---|
| 00012340H | |
| 00012341H | |
| 00012342H | |
| 00012343H | |
| 00012344H | |
| 00012345H | |
| 00012346H | F8H |
| 00012347H | |
| 00012348H | |
| 00012349H | |
| 0001234AH | |
| 0001234BH | |
| 0001234CH | |
| 0001234DH | |
| 0001234EH | |
| 0001234FH | |
| 00012350H | |

主存的基本存储单位是位（BIT）

F 8

| 1 | 1 | 1 | 1 | 1 | 0 | 0 | 0 |
|---|---|---|---|---|---|---|---|

8个位组成一个字节 BYTE

Q: 1M字节内存，地址编码需要多少二进制位？

Q:32位地址对应的内存大小可达到多大？

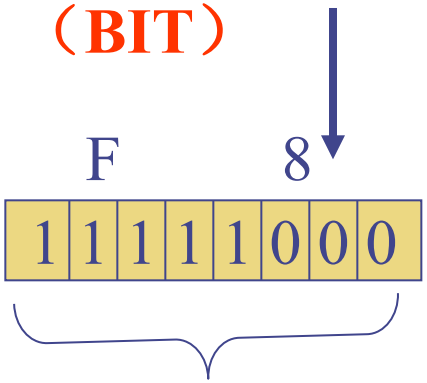
字节是最小的寻址单位

| | |
|---|---|
| 00012340H | |
| 00012341H | |
| 00012342H | |
| 00012343H | |
| 00012344H | |
| 00012345H | |
| 00012346H | F8H |
| 00012347H | 04H |
| 00012348H | 56H |
| 00012349H | 12H |
| 0001234AH | |
| 0001234BH | |
| 0001234CH | |
| 0001234DH | |
| 0001234EH | |
| 0001234FH | |
| 00012350H | |

字地址是这2个字节中低字节的地址

两个相邻的字节组成一个字 WORD

主存的基本存储单位是位（BIT）




8个位组成一个字节 BYTE

**Q1:**两个黄色的字节组成的字的地址是多少？字中的内容又是多少？

**PA:00012346H**
**DATA: 04F8H**

**Q2:**红色的呢？

**PA:00012347H**
**DATA: 5604H**

| | |
|---|---|
| 00012340H | |
| 00012341H | |
| 00012342H | |
| 00012343H | |
| 00012344H | |
| 00012345H | |
| 00012346H | F8H |
| 00012347H | 04H |
| 00012348H | 56H |
| 00012349H | 12H |
| 0001234AH | |
| 0001234BH | |
| 0001234CH | |
| 0001234DH | 78H |
| 0001234EH | 56H |
| 0001234FH | |
| 00012350H | |

字数据的存放形式:

低8位在低字节；高8位在相邻的高字节中。

**Q3:**将5678H存放到地址为1234D的字单元中。

地址为12346H的双字是：
44434241H
即(00012346H) = 44434241H

地址为12346H的字是：
4241H
即(00012346H) = 4241H

地址为12346H的字节是：
41H
即(00012346H) = 41H

| | |
|---|---|
| 00012340H | |
| 00012341H | |
| 00012342H | |
| 00012343H | |
| 00012344H | |
| 00012345H | |
| 00012346H | 41H |
| 00012347H | 42H |
| 00012348H | 43H |
| 00012349H | 44H |
| 0001234AH | |
| 0001234BH | |
| 0001234CH | |
| 0001234DH | |
| 0001234EH | |
| 0001234FH | |
| 00012350H | |

双字
四个连续的字节组成

其地址为四个字节中的最低字节的地址。

# 3.1.1 数据存储的基本形式



数据存储方法有两种

- 小端存储（Little Endian）
- 大端存储（Big Endian）

➢ Intel x86 系列采用小端存储方式

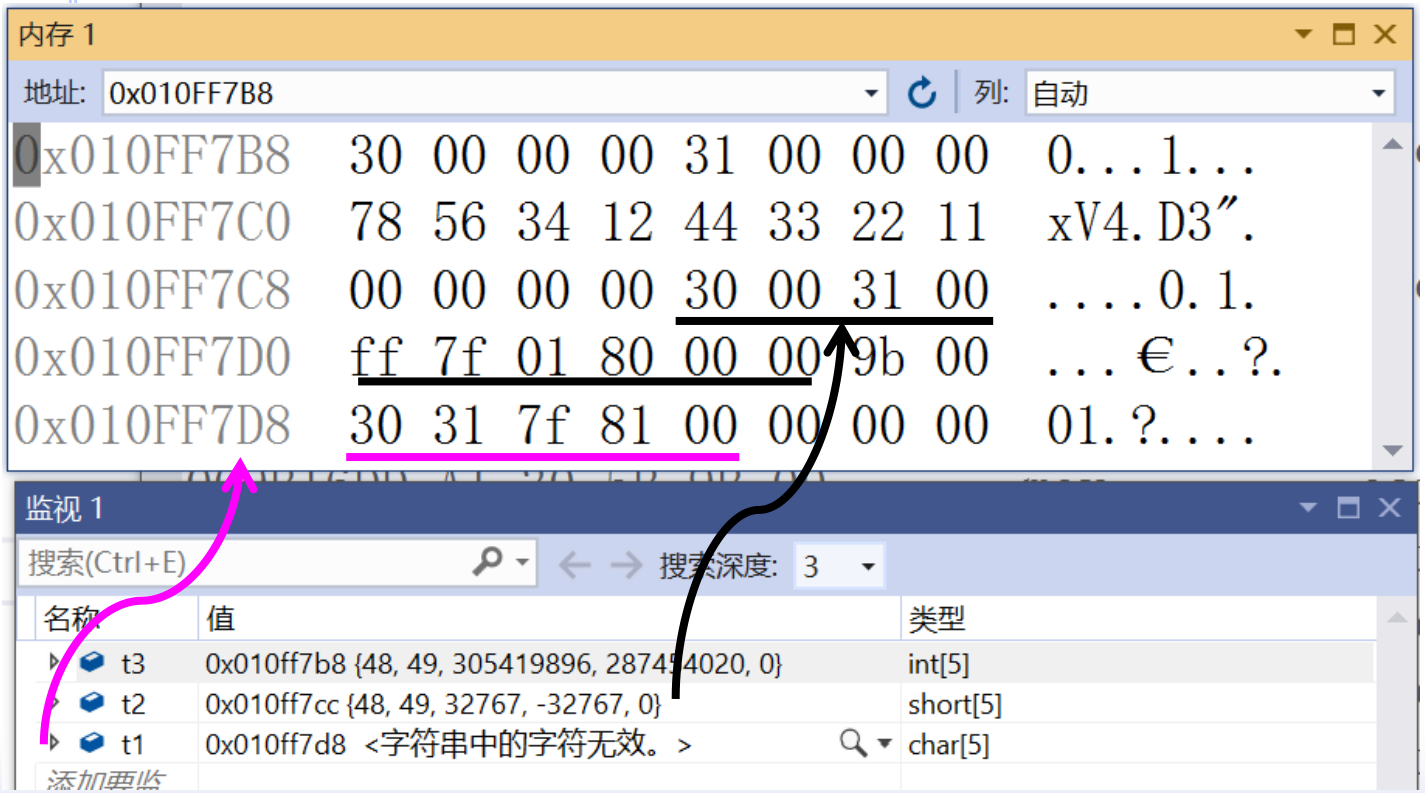
➢ 在小端存储方式中，最低地址字节中存放数据的最低字节，最高地址字节中存放数据的最高字节。按照数据由低字节到高字节的顺序依次存放在从低地址到高地址的单元中。

➢ 大端存储方式与小端存储方式相反。

# 3.1.1 数据存储的基本形式



```
char  t1[5] = { 48, 49, 127, 129, 0 };
short t2[5] = { 48, 49, 32767,  32769, 0 };
int   t3[5] = { 48, 49, 0x12345678, 0x11223344, 0 };
```




要点：

(1) 变量对应的地址，确定存储位置

(2) 数据的表示

(3) 数据存放规则
每个元素所占的字节数；
小端存放

# 3.1.1 数据存储的基本形式

華中科技大學

```
char  t1[5] = { 48,49,127,129,0 };
short t2[5] = { 48,49,32767,  32769,0 };
int   t3[5] = { 48,49,0x12345678,0x11223344,0 };
int   *pt1 = t1; // 警告: 从“char *”到“int *”的类型不兼容
int   q = *pt1;
```

q=?

int * pt1 = (int *)t1;

内存 1

地址: 0x010FF7D8　　列: 自动

| | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|
| 0x010FF7D8 | 30 | 31 | 7f | 81 | 00 | 00 | 00 | 00 | 01.?.... |
| 0x010FF7E0 | ab | 23 | 05 | 2d | 04 | f8 | 0f | 01 | ?#.-.?.. |

监视 1

搜索(Ctrl+E)　　搜索深度: 3

| 名称 | 值 | 类型 |
|---|---|---|
| pt1 | 0x010ff7d8 {-2122370768} | int * |
| q | -2122370768 | int |
| t1 | 0x010ff7d8 <字符串中的字符无效。> | char[5] |
| q,x | 0x817f3130 | int |
| &pt1 | 0x010ff7b4 {0x010ff7d8 {-2122370768}} | int * * |

# 3.1.2 数据地址的类型及转换



给定一个地址后，可以根据该地址取一个字节、一个字、一个双字……　取多少字节的数据，取决于地址类型。

以 … 为字节地址

以 … 为字地址

以 … 为双字地址

C 语言程序中，可以灵活的进行地址类型转换，从而访问同一位置的不同长度的数据。

# 3.1.2 数据地址的类型及转换

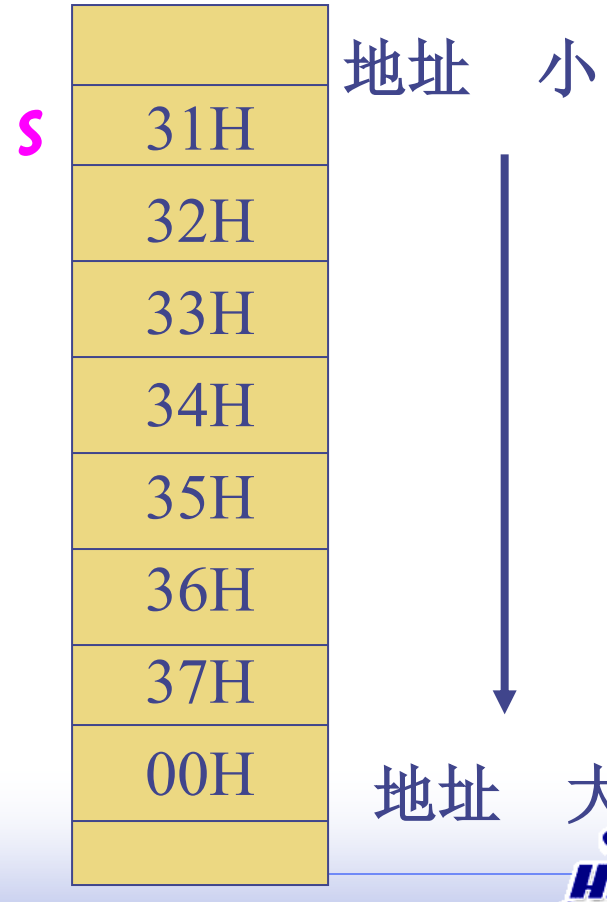


程序运行结果是什么？为什么？

```
char   s[10];
strcpy(s, ”1234567”);
printf(“%x \n”,  *(long *)(s+2));
printf(“%x \n”,  *(short *)(s+2));
printf(“%d \n”,  *(char *)(s+2));
*(int *)(s+1)=16706;
printf(“%s \n”,s);
```

C:\新书示例\C

```
36353433
3433
 51
1BA
```




关键词：地址类型转换

# 3.1.2 数据地址的类型及转换



```
char   s[10];
strcpy(s, ”1234567”);
```

## 地址类型转换

内存 1

地址: 0x00AFFA18　　列: 自动

```
0x00AFFA18   31 32 33 34 35 36 37 00   1234567.
0x00AFFA20   06 90 00 01 96 9d d1 3a   .?..???:
```

监视 1

搜索(Ctrl+E)　　搜索深度: 3

| 名称 | 值 | 类型 |
|---|---|---|
| s | 0x00affa18 "1234567" | char[10] |
| s[2] | 51 '3' | char |
| *(short *)(s+2) | 13363 | short |
| *(short *)(s+2),x | 0x3433 | short |
| *(int *)(s+2),x | 0x36353433 | int |

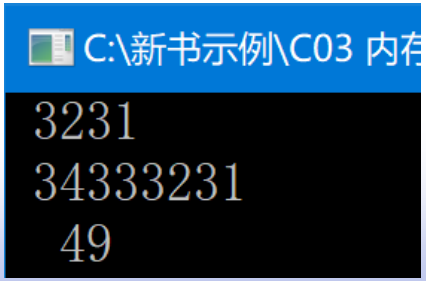
在调试窗口，观察地址类型转换后取数的结果

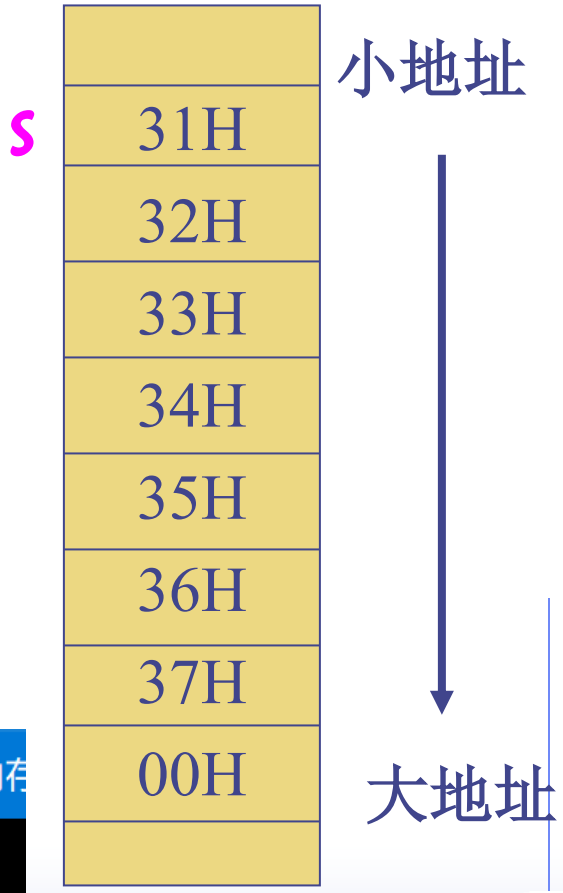
# 3.1.2 数据地址的类型及转换



## 地址类型转换

```
#define _CRT_SECURE_NO_WARNINGS
#include <iostream>
union test {
    char s[10];
    char c;
    short x;
    int   y;
} temp;
int main()
{    strcpy(temp.s, "1234567");
    printf(" % x \n", temp.x);
    printf(" % x \n", temp.y);
    printf(" % d \n", temp.c);
    return 0;
}
```



# 3.1.2 数据地址的类型及转换



## 地址类型转换

```
union test {
    char s[10];
    char c;
    short x;
    int   y;
}temp;
strcpy(temp.s, "1234567");
```

监视 1

搜索(Ctrl+E)　搜索深度: 3

| 名称 | 值 | 类型 |
|---|---|---|
| &temp.x | 0x0092a138 {union_type.exe!test temp} {12849} | short * |
| &temp.y | 0x0092a138 {union_type.exe!test temp} {875770417} | int * |
| &temp.c | 0x0092a138 "1234567" | char * |
| temp.s | 0x0092a138 "1234567" | char[10] |

# 3.1.2 数据地址的类型及转换



```
#include <stdio.h>
struct  struct_test {
    char    c1;
    double  d8;
    short   s2;
    int     i4;
};
int main()
{   struct_test s = { '1',10.5, 20,30 };
    printf("sizeof struct : %d\n", sizeof(s));
    printf("address %p %p %p %p \n",
            &s.c1, &s.d8 , &s.s2, &s.i4 );
    return 0;
}  //s大小等同 sizeof(struct_test);
```

**结构变量的空间分配**

内存 1

地址: 0x00B3F754

| | | | | | | | | |
|---|---|---|---|---|---|---|---|---|
| 0x00B3F754 | 31 | cc | cc | cc | cc | cc | cc | cc |
| 0x00B3F75C | 00 | 00 | 00 | 00 | 00 | 00 | 25 | 40 |
| 0x00B3F764 | 14 | 00 | cc | cc | 1e | 00 | 00 | 00 |

C:\教学\本科教学\计算机系统基础\计算机系统基础_程序\C03_数据的机内表示\数据的机内表示\Debug\struct_test.exe

```
sizeof struct : 24
address 00B3F754 00B3F75C 00B3F764 00B3F768
```

# 3.1.2 数据地址的类型及转换



```
#include <stdio.h>
#pragma pack(1)
struct  struct_test {
    char    c1;
    double  d8;
    short   s2;
    int     i4;
};
int main()
{   struct_test s = { '1',10.5, 20,30 };
    printf("sizeof struct : %d\n", sizeof(s));
    printf("address %p %p %p %p \n",
           &s.c1, &s.d8 , &s.s2, &s.i4 );
    return 0;
}  //s大小等同 sizeof(struct_test);
```

结构变量的空间分配

不同的编译开关分配的结果不同！

```
sizeof struct : 15
address 008FF758 008FF759 008FF761 008FF763
```

# 3.1.2 数据地址的类型及转换



## 以不同形式观察内存

- 没有数据(D)
- ✓ 1 字节整数(1)
- 2 字节整数(2)
- 4 字节整数(4)
- 8 字节整数(8)
- 32 位浮点(3)
- 64 位浮点(6)
- ✓ 十六进制显示(H)
- 带符号显示(S)
- 不带符号显示(U)
- Big Endian(B)
- 没有文本(N)
- ✓ ANSI 文本(A)
- Unicode 文本(I)
- 复制(Y) Ctrl+C
- 编辑值(E)
- 自动重新计算(R)
- ✓ 显示工具栏(T)

| | | | | | | | | |
|---|---|---|---|---|---|---|---|---|
| 0x0133FAEC | 31 | cc | cc | cc | cc | cc | cc | cc |
| 0x0133FAF4 | 00 | 00 | 00 | 00 | 00 | 00 | 25 | 40 |
| 0x0133FAFC | 14 | 00 | cc | cc | 1e | 00 | 00 | 00 |

| | | | | |
|---|---|---|---|---|
| 0x0133FAEC | cc31 | cccc | cccc | cccc |
| 0x0133FAF4 | 0000 | 0000 | 0000 | 4025 |
| 0x0133FAFC | 0014 | cccc | 001e | 0000 |

| | | |
|---|---|---|
| 0x0133FAEC | cccccc31 | cccccccc |
| 0x0133FAF4 | 00000000 | 40250000 |
| 0x0133FAFC | cccc0014 | 0000001e |

# 3.1.2 数据地址的类型及转换



**练习：体会地址类型转换的应用**

定义了 结构 student， 以及结构数组变量s[3];

```
struct student {
    char  name[8];
    short age;
    float  score;
    char  remark[200];
};
student  s[3];
student  new_s[3];
```

编写程序，将 s[3] 中的信息紧凑存放到 一个字符数组 message 中，然后从 message 转换到结构数组 new_s[3]中。

# 3.1.2 数据地址的类型及转换



练习

Microsoft Visual Studio 调试控制台

```
结构 student的大小 =214

结构数组 s 的大小 =642

张三   20   91.5   2021年获得三好学生称号,2020年获得优秀干部奖
xuxiang   21   94.7   very good student
wang   22   87.6   none
packed :103

张三   20   91.5   2021年获得三好学生称号,2020年获得优秀干部奖
xuxiang   21   94.7   very good student
wang   22   87.6   none
```

要求 s[0].name 为自己的姓名;
可以在给的程序中 补充

void pack_student(student* s)
void restore_student(student* s)

# 小结

- 整型数据的存放规律：长度与类型有关、小端存放；
- 内存中存放的数据串是一个字节流；
- 从同一个地址开始，用不同类型来解读，得到不同结果；

  [从一个地址开始的 char、short、int、float、double ]

  [ unsigned char、unsigned short、unsigned int ]
- 每个变量都有一个地址；
- 变量地址的表达形式与编译器有关；
- 不同的编译开关可生成不同的地址表达形式。

# 3.2 数值数据在计算机内的表示形式



- 数据
  - 数值数据
    - 定点表示法
    - 浮点表示法（实数）
  - 字符数据

常用的数值数据为二进制、十进制、十六进制、BCD码。

字符数据：ASCII（美国信息标准交换代码）

GB2312

GBK

# 3.2.1 有符号和无符号整数的表示形式



Q: 设 char sa[5] = {-100, 156, -255, 1, ‘a’, 97, 0};

unsigned char usa[5]= {-100, 156, -255, 1,’a’,97, 0};

数组sa, usa中存放的结果是什么？为什么？

监视 1

搜索(Ctrl+E) 搜索深度: 3

| 名称 | 值 |
|---|---|
| sa | 0x003cf6f4 "潜\x1\x1aa" |
| [0] | -100 '?' |
| [1] | -100 '?' |
| [2] | 1 '\x1' |
| [3] | 1 '\x1' |
| [4] | 97 'a' |
| [5] | 97 'a' |
| [6] | 0 '\0' |
| usa | 0x003cf6ec "潜\x1\x1aa" |

数据的存放与unsigned 无关。但是相同字节流解读成有符号或者无符号数，是不同的。

sa[0] = 0x9c > 0 不成立

usa[0] = 0x9c > 0 成立！

内存 1

地址: 0x003CF6EC 列: 自动

```
0x003CF6EC  9c 9c 01 01 61 61 00 00  ??..aa..
0x003CF6F4  9c 9c 01 01 61 61 00 00  ??..aa..
0x003CF6FC  38 58 46 b6 20 f7 3c 00  8XF? ?<.
0x003CF704  b3 1c 84 00 01 00 00 00  ? ?
```

# 3.2.1 有符号和无符号整数的表示形式



■ 有符号数的表示范围

8位补码： 80H --- 7FH (-128 ,127)

16位补码： 8000H --- 7FFFH (-32768 , 32767)

正数

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| 0 | 0 | 0 | 0 | 0 | 0 | 0 | 1 |

1

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| 0 | 1 | 1 | 1 | 1 | 1 | 1 | 1 |

7F

N=8

负数

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 |

-1

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| 1 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |

-128

# 3.2.1 有符号和无符号整数的表示形式



- 有符号数（补码表示）的大小比较

设 n = 16, 试比较：

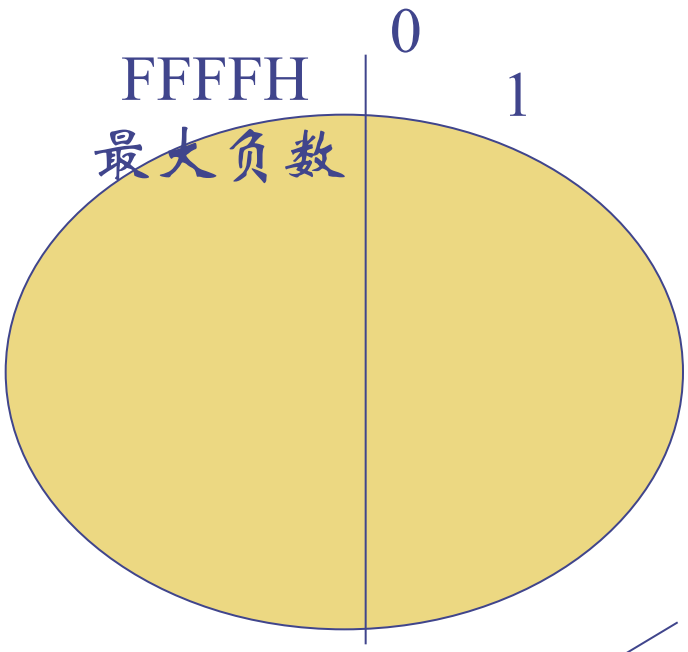
| | | | |
|---|---|---|---|
| 5678H | vs | 7345H ? | < |
| 5678H | vs | 8345H ? | > |
| 3271H | vs | 0A521H ? | > |
| 0A521H | vs | 0B521H ? | < |

# 3.2.1 有符号和无符号整数的表示形式



■ 有符号数（补码表示）的大小比较

5678H < 7345H
5678H > 8345H
3271H > 0A521H
0A521H < 0B521H




最小负数 → 8000H

7FFFH ← 最大正数

# 3.2.1 有符号和无符号整数的表示形式



- 无符号数及其表示范围

一个段中的偏移地址，就是一个无符号数。
最小的无符号数是0。

8位无符号数：　0H --- FFH　(0 ,255)

16位无符号数：　0H --- FFFFH　(0 ,65535)

无符号数的大小比较？

# 3.2.1 有符号和无符号整数的表示形式



设 n=8, 有一个数 M,

| | | | | | | | |
|---|---|---|---|---|---|---|---|
| 1 | 1 | 1 | 1 | 1 | 1 | 1 | 1 |

**Question:** 若将其看成一个数的补码，它表示的什么数？

-1

**Question:** 如果M作为无符号数，它表示的是多少？

255

**Question:** 哪到底将它看成什么数呢？

**Answer:** 这取决于访问该单元的指令。

有 / 无 unsigned 的变量, 编译生成的机器指令不同。

# 3.2.1 有符号和无符号整数的表示形式



```
#include <stdio.h>

void main( )

{
    short   x;
    unsigned short  y;
    x= -1;          // x=65535
    y= -1;          // y=65535
    printf("%d   %d\n", x,  y);

}
```

结果是：
-1  65535

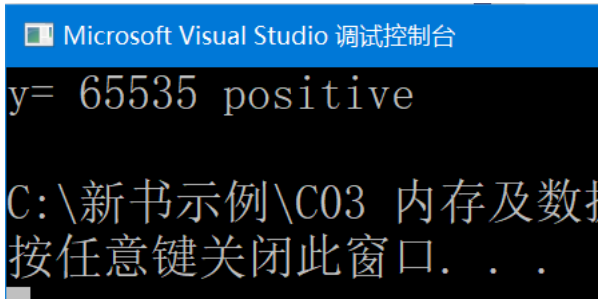
在汇编语言中也有类似于unsigned的约定。

# 3.2.1 有符号和无符号整数的表示形式



```
#include <stdio.h>
void main( )
{
    short   x;
    unsigned short  y;
    x = -1;    y = -1;
    if (x > 0)
        printf("x= %d positive\n",x);
    if (y > 0)
        printf("y= %d positive\n",y);
}
```

结果？




谁来把一个数当有符号数看，还是无符号数？

# 3.2.1 有符号和无符号整数的表示形式



```
    if (x > 0)  printf("x= %d positive\n",x);
001D1875  movsx       eax,word ptr [x]
001D1879  test        eax,eax
001D187B  jle         __$EncStackInitStart+43h (01D188Fh)
001D187D  movsx       eax,word ptr [x]
001D1881  push        eax
001D1882  push        offset string "x= %d positive\n" (01D7B30h)
001D1887  call        _printf (01D10CDh)
001D188C  add         esp,8
    if (y > 0)  printf("y= %d positive\n", y);
001D188F  movzx       eax,word ptr [y]
001D1893  test        eax,eax
001D1895  jle         __$EncStackInitStart+5Dh (01D18A9h)
001D1897  movzx       eax,word ptr [y]
001D189B  push        eax
001D189C  push        offset string "y= %d positive\n" (01D7B44h)
001D18A1  call        _printf (01D10CDh)
001D18A6  add         esp,8
```

# 小结



- 整型数据的表示与unsigned 无关。
- 不论是有符号整数还是无符号整数，都是采用补码表示。
- 同一个数字，当成有符号数解读与当成无符号数解读，解读的结果可能不同。
- C 语言中，char/short/int、unsigned char/short/int 是不同的类型；对于比较运算，生成的机器指令不同。

# 3.2.2 BCD码

BCD (Binary Coded Decimal):二进制编码的十进制

用4位二进制数表示一位十进制数。

1 = 0001 BCD　　8 = 1000 BCD

2 = 0010 BCD　　9 = 1001 BCD

……

9781的压缩BCD码

| |
|---|
| 1000 0001 |
| 1001 0111 |

| |
|---|
| 0000 0001 |
| 0000 1000 |
| 0000 0111 |
| 0000 1001 |

9781的非压缩BCD码

# BCD码 扩展知识



BCD码可分为两类：有权BCD码和无权BCD码

有权BCD码：8421码、2421码、5421码

无权BCD码：余3码、余3循环码、格雷码

- 8421码：从高位到低位的权值为8、4、2、1，和四位自然二进制码不同的是，它只选用了四位二进制码中前10组代码，即用0000~1001分别代表它所对应的十进制数，余下的六组代码1010~1111不用。
- 2421码：从高位到低位的权值为2、4、2、1，注意：数码6，既可以用1100表示， 也可以用0110表示
- 5421码：从高位到低位的权值分别为5、4、2、1，注意：数码5，既可以用1000表示，也可以用0101表示

- 余3码：在8421 BCD码的每基础上加3(0011)形成的，比如8->1011,9->1100
- 余3循环码：余3循环码是变权码，每一位的1并不代表固定的数值，它具有格雷码的优点，即两个相邻代码之间仅有一位的状态不同。
- 格雷码：在一组数的编码中，若任意两个相邻的代码只有一位二进制数不同，则称这种编码为格雷码（Gray Code），另外由于最大数与最小数之间也仅一位数不同，即“首尾相连”，因此又称循环码或反射码。

# 格雷码 Gray Code

華中科技大學

| 十进制数 | 自然二进制数 | 格雷码 |
|---|---|---|
| 0 | 0000 | 0000 |
| 1 | 0001 | 0001 |
| 2 | 0010 | 0011 |
| 3 | 0011 | 0010 |
| 4 | 0100 | 0110 |
| 5 | 0101 | 0111 |
| 6 | 0110 | 0101 |
| 7 | 0111 | 0100 |
| 8 | 1000 | 1100 |
| 9 | 1001 | 1101 |
| 10 | 1010 | 1111 |
| 11 | 1011 | 1110 |
| 12 | 1100 | 1010 |
| 13 | 1101 | 1011 |
| 14 | 1110 | 1001 |
| 15 | 1111 | 1000 |

二进制码->格雷码（编码）：

- 最左边一位不变；
- 从左位第二位起，依次将每一位与左边一位异或（XOR），作为对应格雷码在该位的值。
- 相邻的数字编码，只有一个二进制位发生变化
- 常用于模拟－数字转换中

# 余3码



| 十进制数 | 8421码 | 余3码 |
|---|---|---|
| 0 | 0000 | 0011 |
| 1 | 0001 | 0100 |
| 2 | 0010 | 0101 |
| 3 | 0011 | 0110 |
| 4 | 0100 | 0111 |
| 5 | 0101 | 1000 |
| 6 | 0110 | 1001 |
| 7 | 0111 | 1010 |
| 8 | 1000 | 1011 |
| 9 | 1001 | 1100 |

以8421码（即BCD）为基础，
每位十进制数BCD码，加上0011得到。

# 3.3 字符数据在机内的表示形式



## ASCII码字符表

| | | 0 | 16 | 32 | 48 | 64 | 80 | 96 | 112 | 128 | 144 | 160 | 176 | 192 | 208 | 224 | 240 |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| ↓ | | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | A | B | C | D | E | F |
| 0 | 0 | | | | **0** | | **P** | | **p** | | | | | | | | |
| 1 | 1 | | | | **1** | **A** | **Q** | **a** | **q** | | | | | | | | |
| 2 | 2 | | | | **2** | **B** | **R** | **b** | **r** | | | | | | | | |
| 3 | 3 | | | | **3** | **C** | **S** | **c** | **s** | | | | | | | | |
| 4 | 4 | | | **$** | **4** | **D** | **T** | **d** | **t** | | | | | | | | |
| 5 | 5 | | | | **5** | **E** | **U** | **e** | **u** | | | | | | | | |
| 6 | 6 | | | | **6** | **F** | **V** | **f** | **v** | | | | | | | | |
| 7 | 7 | | | | **7** | **G** | **W** | **g** | **w** | | | | | | | | |
| 8 | 8 | | | | **8** | **H** | **X** | **h** | **x** | | | | | | | | |
| 9 | 9 | | | | **9** | **I** | **Y** | **i** | **y** | | | | | | | | |
| 10 | A | 換行 | | | | **J** | **Z** | **j** | **z** | | | | | | | | |
| 11 | B | | | | | **K** | | **k** | | | | | | | | | |
| 12 | C | | | | | **L** | | **l** | | | | | | | | | |
| 13 | D | 回车 | | | | **M** | | **m** | | | | | | | | | |
| 14 | E | | | | | **N** | | **n** | | | | | | | | | |
| 15 | F | | | | | **O** | | **o** | | | | | | | | | |

American Standard Code for Information Interchange

# 3.3 字符数据在机内的表示形式

字符串
“1234ABCD”
的表示结果：

以0 为串的结束

也可以写成’\0’

| |
|---|
| 31H |
| 32H |
| 33H |
| 34H |
| 41H |
| 42H |
| 43H |
| 44H |
| 00 |

地址　小

↓

地址　大

# 3.3 字符数据在机内的表示形式



## 汉字编码 - GB2312

- 简体中文字符集，含6763个汉字和682个非汉字（图形符号），双字节编码。
- 所有字符组成一个94x94的矩阵。矩阵的每一行称为一个“区”，每一列称为一个"位"（行列号称为区位号）。

- 01-09区为682个非汉字字符（图形符号）
- 16-55区为一级汉字（3755个最常用的汉字，按拼音字母顺序排列）
- 56-87区为二级汉字（3008个汉字，按部首次序排列）
- 10-15区、88-94是空白区（保留区）。

## 汉字编码 - GB2312

区位码：由行号（区号）为高字节、列号（位号）位低字节组成的字

国际码（交换码）：区位码 + 0x2020（似乎没什么用）

机内码：区位码 + 0xA0A0

‘华’：区位码 = 27 10，即 0x1B0A

国际码 = 0x3B2A

机内码 = 0xBBAA（计算机存储）

# 3.3 字符数据在机内的表示形式



char s[] = "123华中科技大学";

内存 1

地址: 0x009FF998　　列: 自动

| | | | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|
| 0x009FF998 | 31 | 32 | 33 | bb | aa | d6 | d0 | bf | 123????? |
| 0x009FF9A0 | c6 | bc | bc | b4 | f3 | d1 | a7 | 00 | ???????. |

char16_t　hz = '华';

监视 1

搜索(Ctrl+E)　搜索深度: 3

| 名称 | 值 | 类型 |
|---|---|---|
| hz | 48042 u'뺪' | char16_t |
| hz,x | 0xbbaa u'뺪' | char16_t |

# 3.3 字符数据在机内的表示形式



**Unicode：**

- 将全世界所有的字符包含在一个集合里；
- 只要支持该字符集，就能显示所有字符，不会有乱码了。
- 从0开始，为每个符号指定一个编号(码点, code point)
  码点U+0041：英语的大写字母A；码点U+534E：汉字“华”
- 在Unicode 7.0版中，收入了109449个符号，其中的中日韩文字为74500个。
- 最前面的65536个字符位，码点从 U+0000 到 U+FFFF。称为基本多文种平面（BMP），含最常见的字符。
  Basic Multilingual Plane

# 3.3 字符数据在机内的表示形式

華中科技大學

```
char16_t  hz  = '华';        //GB2312
char16_t  hz1 = u '华';      //Unicode, UTF-16
```

| | | |
|---|---|---|
| hz | 48042 u'뿪' | char16_t |
| hz,x | 0xbbaa u'뿪' | char16_t |
| hz1,x | 0x534e u'华' | char16_t |
| hz1 | 21326 u'华' | char16_t |

猴:29492 吼:21564 厚:21402 候:20505 后:21518 呼:21628 乎:20046 忽:24573 瑚:29786 壶:22774
葫:33899 胡:32993 蝴:34676 狐:29392 糊:31946 湖:28246 弧:24359 虎:34382 唬:21804 护:25252
互:20114 沪:27818 户:25143 花:33457 哗:21719 华:21326 猾:29502 滑:28369

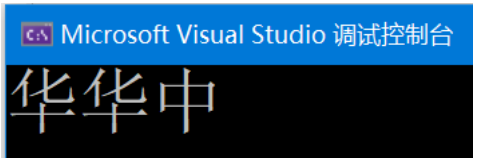
**汉字的 unicode 编码表**

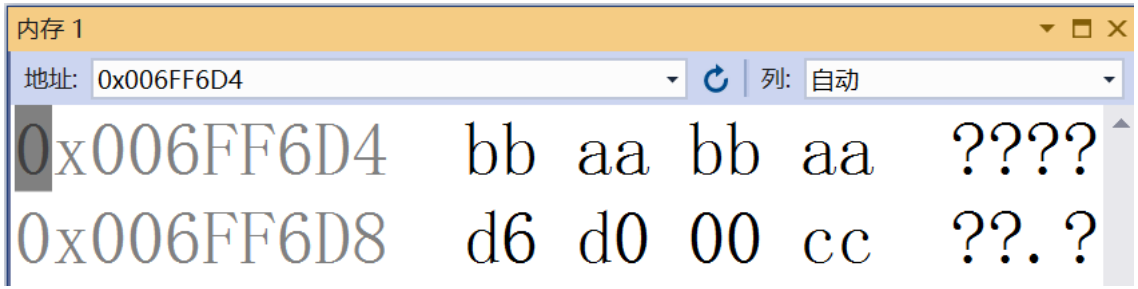
**https://blog.csdn.net/Aimless520/article/details/126692649**

# 3.3 字符数据在机内的表示形式



```
char  p[] = "华\u534e中";
cout << p << endl;
```



# 3.3 字符数据在机内的表示形式



Unicode存在的问题：

- 如果所有字符都用2个字节、3个字节甚至更多字节来表示，会造成存储空间的增加，太费存储空间
- ASCII只需要一个字节来进行编码
- 如何区别该编码是Unicode还是ASCII？

- 如何节约空间？
- 如何避免对编码误判？

**Unicode 的实现方式：** UTF-8、UTF-16、UTF-32

Unicode Transformation Format， 转换格式

# 3.3 字符数据在机内的表示形式



UTF-8

- 是一种变长的编码方法，长度从1个到4个字节不等；
- 越是常用的字符，字节越短；
- 对于单字节的符号

  字节的第一位设为0，后面7位为这个符号的Unicode码。

  对于英文字母，UTF-8 编码和 ASCII 编码是相同的。
- 对于n字节的符号（n>1）

  第一个字节的前n位都设为1，第n+1位设为0，

  后面字节的前两位一律设为10，

  剩下没有提及的二进制位，全为这个符号的 Unicode 码。

# 3.3 字符数据在机内的表示形式



- 对于n字节的符号（n>1）

  第一个字节的前n位都设为1，第n+1位设为0，

  后面字节的前两位一律设为10，

  剩下没有提及的二进制位，全为这个符号的 Unicode 码。

码点U+534E： 汉字“华” → UTF-8

0101 0011 0100 1110

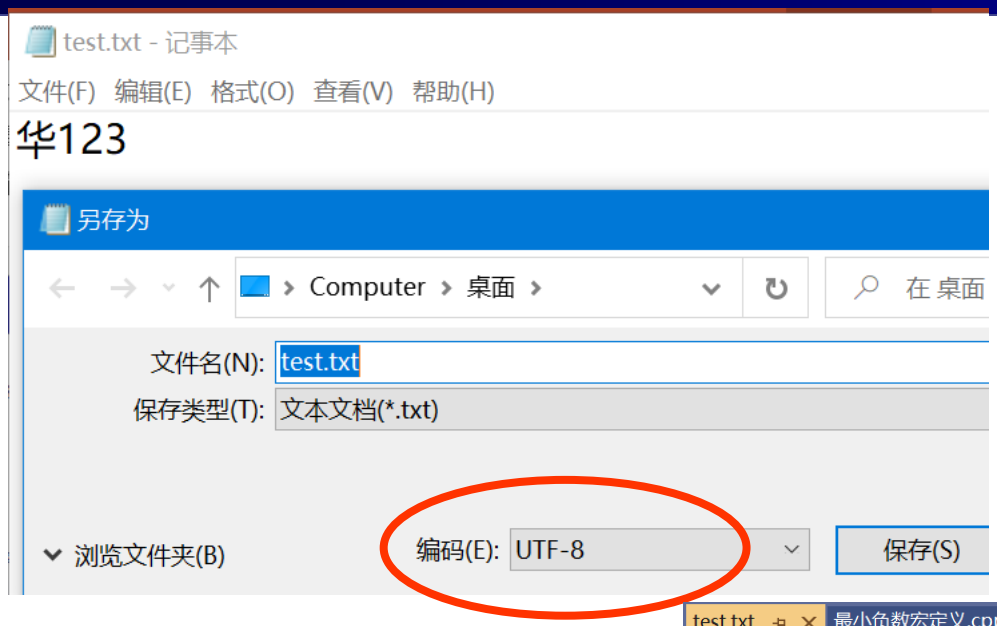
15个有效二进制位，用3个字节来表示

1110 0101 10 00 11 01 10 00 1110

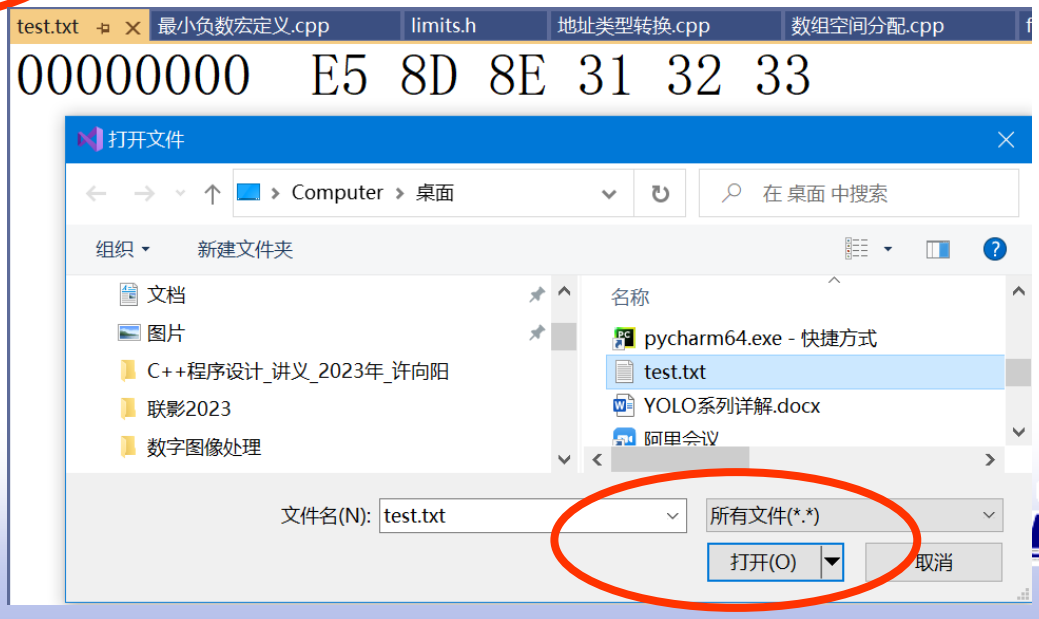
E5 8D 8E

# 3.3 字符数据在机内的表示形式






保存文件时，可以选择编码方式




打开方式中，以2进制形式打开

# 3.3 字符数据在机内的表示形式



UTF-16

- 介于UTF-8和UTF-32之间，结合了定长和变长两种编码方式的特点；
- 基本平面的字符用2个字节，辅助平面的字符用4个字节。
- 2个字节（U+0000 到 U+FFFF）

  UTF-16编码就是码点 对应的16位无符号整数
- 4个字节（U+010000 到 U+10FFFF）

  先计算 U’ = U - 0x10000

  将U’写成二进制形式：yyyy yyyy yyxx xxxx xxxx，

  U的UTF-16编码：110110yyyyyyyyyy 110111xxxxxxxxxx。

# 3.3 字符数据在机内的表示形式



UTF-32

- UTF-32是最直观的编码方法
- 每个码点使用四个字节表示
- 字节内容一一对应码点
- 转换规则简单直观，查找效率高。缺点在于浪费空间
- 同样内容的英语文本，它会比ASCII编码大四倍。
- 实际上没有人使用这种编码方法

  HTML 5标准就明文规定，网页不得编码成UTF-32。

# 3.3 字符数据在机内的表示形式



编码(E): GB18030

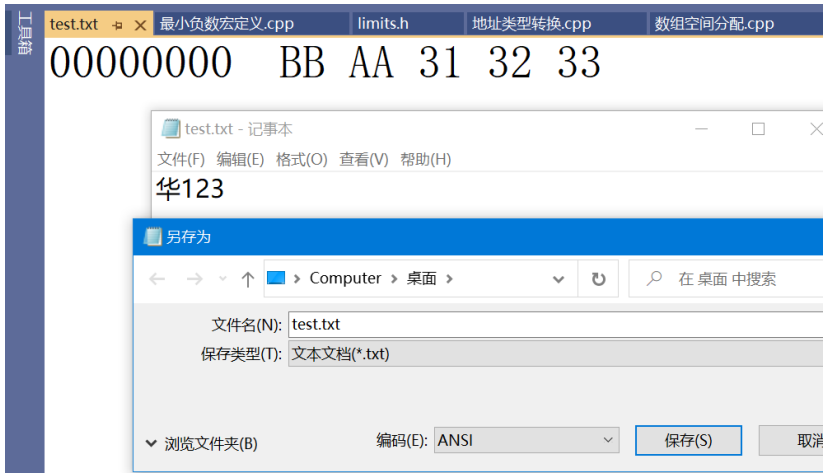
## ANSI 编码

- 不同的国家和地区制定了不同的标准
- 在简体中文Windows操作系统中，ANSI编码代表GB编码;
- 在日文Windows系统中，ANSI编码代表 Shift_JIS 编码。
- 不同 ANSI 编码之间互不兼容;
- 对于ANSI编码而言，0x00～0x7F之间的字符，依旧是1个字节代表1个字符。

American National Standards Institute 美国国家标准学会

# 3.3 字符数据在机内的表示形式






在 GB2312 中，“华”的区位码是 2710，第27区的第10个字（共 94区，每区 94个字）（1B 0A）。

区位码的每个字节加 A0，得到 ANSI 编码，即 BB AA。

为了保证 Windows 在不同语言文字的国家都能用，微软采用了标准代码页 (Code Page，代码页是字符集编码的别名，也叫内码表) 的方法，把全世界的编码方式统一编号，在不同的地方采用对应地方的编码方式，微软把它称为 ANSI 编码。

» chcp

活动代码页 936

```
C:\WINDOWS\System32>chcp
活动代码页: 936
```

# 3.3 字符数据在机内的表示形式

# 3.3 字符数据在机内的表示形式



**Q:** 在键盘上输出123。计算机中得到是什么呢？若要用其作数值运算，如何办呢？

“123”的字符串形式　　123的数值形式

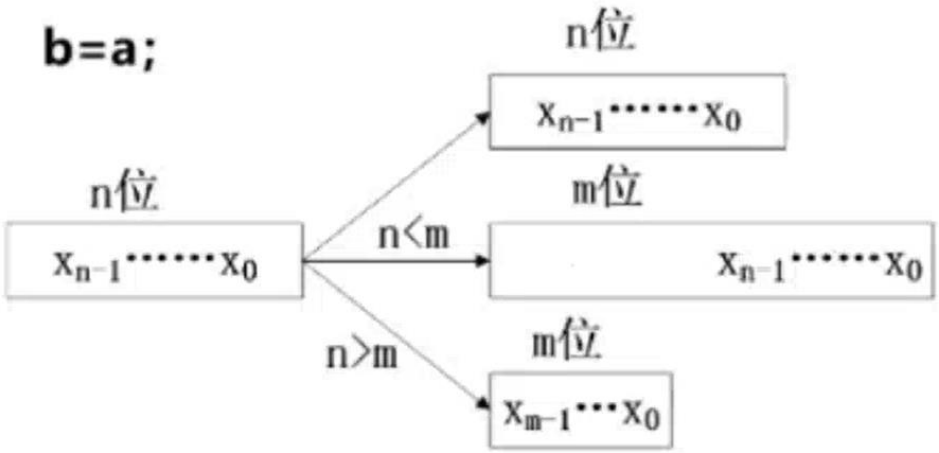
# 3.4 数据类型转换



- 不同整数类型的变量赋值，如何进行转换？

**a和b都是整数**

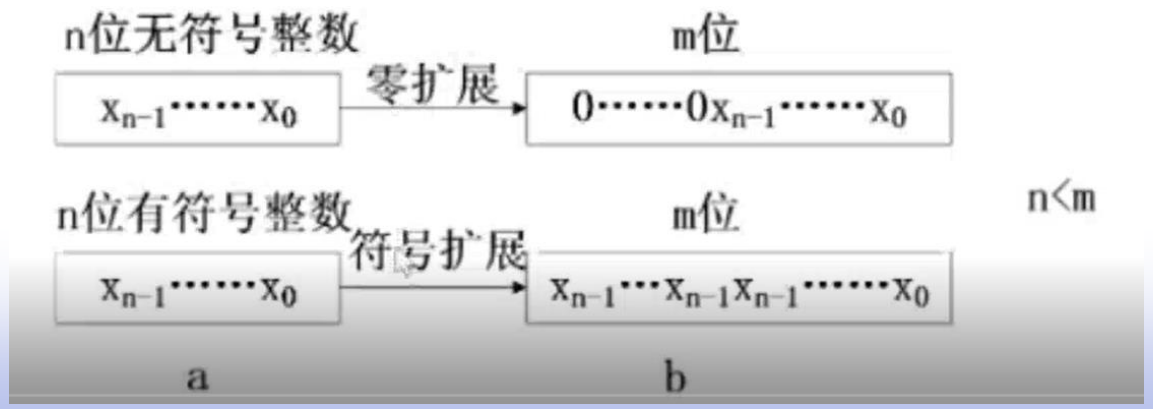
**b=a;**




**机器数上的复制**

**符号扩展或零扩展**

**截断**




n<m

# 3.4 数据类型转换



- 不同整数类型的变量赋值，如何进行转换？

**规则1：两边的长度一样，直接复制，与类型无关；**
**规则2：右边长，左边短，截断，保留低位；**
**规则3：右边短，左边长，右边的数需要扩展。**

**Q**：在需要扩展时，何时用符号扩展？
何时用 0 扩展？

```
int  i1;   short s1;   char c1;   bool  b1;
unsigned int  i2;  unsigned short s2;   unsigned char c2;
```

# 3.4 数据类型转换



```
short s1 = 0xffff;
unsigned short s2 = 0xffff;

int  i1;
i1 = s1;    // i1=0xffffffff;

i1 = s2;    // i1=0x0000ffff;
```

```
movsx eax,word ptr [s1]
mov   dword ptr [i1], eax

movzx eax,word ptr [s1]
mov   dword ptr [i1], eax
```

short → int =>赋值给 int

unsigned short → unsigned int =>赋值给 int

结论：以右边的类型进行扩展

# 3.4 数据类型转换



```
short s1 = 0xffff;                s1=-1;
unsigned short s2 = 0xffff;       s2=-1;
int  i1;      unsiged int  i2;

i1 = s1;    // i1=0xffffffff;
i1 = s2;    // i1=0x0000ffff;

i2 = s1;    // i2=0xffffffff;
i2 = s2;    // i2=0x0000ffff;
```

结论：以右边的类型进行扩展；
与 赋值号 左边变量的类型无关

# 3.5 浮点数据在机内的表示形式



```
    float f1 = 1.25;
00101865 F3 0F 10 05 40 7
0010186D F3 0F 11 45 F8
    double d1 = 1.25;
00101872 F2 0F 10 05 48 7
0010187A F2 0F 11 45 E8
    long double ld1 = 1.2
0010187F F2 0F 10 05 48 7
00101887 F2 0F 11 45 D8
```

内存 1

地址: 0x0059FDD8　　列: 自动

```
0x0059FDD8  00 00 a0 3f cc cc cc cc 00 fe 59 00  ..??
0x0059FDE4  83 20 10 00 01 00 00 00 b0 92 ac 00  ? ..
0x0059FDF0  18 7a ac 00 01 00 00 00 b0 92 ac 00  .z?.
```

监视 1

搜索(Ctrl+E)　　搜索深度: 3

| 名称 | 值 | 类型 |
|---|---|---|
| f1 | 1.25000000 | float |
| &f1 | 0x0059fdd8 {1.25000000} | float * |
| 添加要监视的项 | | |

float　f1=1.25;

对应的4个字节内容为：　3f　a0　00　00

# 3.5 浮点数据在机内的表示形式



```
    double d1 = 1.25;
00101872 F2 0F 10 05 48 7B 10 00 movsd       xmm0,mmword ptr [__real@3ff400000C
0010187A F2 0F 11 45 E8
    long double ld1 = 1.2
0010187F F2 0F 10 05 48 7
00101887 F2 0F 11 45 D8
    printf("%f %f %f \n",
0010188C 83 EC 08
0010188F F2 0F 10 45 D8
00101894 F2 0F 11 04 24
00101899 83 EC 08
```

内存 1

地址: 0x0059FDC8　　列: 自动

```
0x0059FDC8  00 00 00 00 00 00 f4 3f cc cc cc cc  ....
0x0059FDD4  cc cc cc cc 00 00 a0 3f cc cc cc cc  ????
0x0059FDE0  00 fe 59 00 83 20 10 00 01 00 00 00  .?Y.
```

监视 1

搜索(Ctrl+E)　　搜索深度: 3

| 名称 | 值 | 类型 |
|---|---|---|
| f1 | 1.25000000 | float |
| &f1 | 0x0059fdd8 {1.25000000} | float * |
| &d1 | 0x0059fdc8 {1.2500000000000000} | double * |

double　d1=1.25;

对应的8个字节内容为：3f f4 00 00 00 00 00 00

# 3.5 浮点数据在机内的表示形式



$$\pm(a_n 2^n + \cdots + a_1 2^1 + a_0 2^0 + a_{-1} 2^{-1} + \cdots + a_{-m} 2^{-m} + \cdots)$$

1.25 = 1.01B;  = 0.101B *2 = 0.0101B *$2^2$;

**规格化数据：** ±1.XXXX*$2^n$ B

科学表示法：小数点左边恒为1

要素：正负号、尾数 XXXX、指数 n

**问题：尾数 XXXX、指数各用多少位来表示？**

**用什么形式来表达？**

# 3.5 浮点数据在机内的表示形式



| 31 | 30　　23 | 22　　0 |
|---|---|---|
| 符号 | 8 位指数 | 23 位尾数 |

**单精度浮点数 float**

符号位：0表示正数，1表示负数

指　数：采用“移码”来表示。单精度移 7F

-126 D -> 00000001 B

127 D -> 11111110 B

尾　数：用原码表示。对于规格化浮点数，尾数的小数点前面的1，不需存贮。

$1.25 = 1.01B = 1.01 * 2^0$;

0 0111 1111　010 00000000000000000000 B

对应的4个字节内容为：　3f a0 00 00

# 3.5 浮点数据在机内的表示形式



| 63 | 62　　52 | 51　　0 |
|---|---|---|
| 符号 | 11 位指数 | 52 位尾数 |

双精度浮点数
**double**

符号位：0表示正数，1表示负数

指　数：采用“移码”来表示。双精度移 3FF

-1022D -> 000 0000 0001B

1023D -> 111 1111 1110B

double d1=1.25 = 1.01 * $2^0$ ;

0 011 1111 1111 0100 0000 …… 0000

对应的8个字节内容为：3f f4 00 00 00 00 00 00

# 3.5 浮点数据在机内的表示形式



| 31 | 30 23 | 22 0 |
|---|---|---|
| 符号 | 8 位指数 | 23 位尾数 |

单精度浮点数
**float**

| 63 | 62 52 | 51 0 |
|---|---|---|
| 符号 | 11 位指数 | 52 位尾数 |

双精度浮点数
**double**

| 79 | 78 64 | 63 0 |
|---|---|---|
| 符号 | 15 位指数 | 63 位尾数 |

高精度浮点数
**long double**

符号位：0表示正数，1表示负数

指　数：采用“移码”来表示。

单精度移 7F　　双精度移 3FF

高精度移 3FFF

# 3.5 浮点数据在机内的表示形式



对于float

最小的规格化正浮点数是： $1.00...000B * 2^{-126}$

非规格化数据：阶码为 0

比上面数小一丁点的数是： $0.11....111B * 2^{-126}$

继续逐步变小，最后会有： $0.00....001B * 2^{-126}$

再小一丁点，就是： $0.00 ...000B * 2^{-126}$

0 的表示（阶码为0，尾数为0）。

# 3.5 浮点数据在机内的表示形式



| 31 | 30 23 | 22 0 |
|---|---|---|
| 符号 | 8 位指数 | 23 位尾数 |

单精度浮点数：**float**

(1) 规格化浮点数
(2) 非规格化浮点数
(3) 特殊数值（±∞, NaN等）

表 2.2 IEEE 754 浮点数的解释

| 值的类型 | 单精度（32 位） | | | | 双精度（64 位） | | | |
|---|---|---|---|---|---|---|---|---|
| | 符号 | 阶码 | 尾数 | 值 | 符号 | 阶码 | 尾数 | 值 |
| 正零 | 0 | 0 | 0 | 0 | 0 | 0 | 0 | 0 |
| 负零 | 1 | 0 | 0 | −0 | 1 | 0 | 0 | −0 |
| 正无穷大 | 0 | 255(全1) | 0 | ∞ | 0 | 2047(全1) | 0 | ∞ |
| 负无穷大 | 1 | 255(全1) | 0 | −∞ | 1 | 2047(全1) | 0 | −∞ |
| 无定义数（非数） | 0 或 1 | 255(全1) | ≠0 | NaN | 0 或 1 | 2047(全1) | ≠0 | NaN |
| 规格化非零正数 | 0 | $0<e<255$ | $f$ | $2^{e-127}(1.f)$ | 0 | $0<e<2047$ | $f$ | $2^{e-1023}(1.f)$ |
| 规格化非零负数 | 1 | $0<e<255$ | $f$ | $-2^{e-127}(1.f)$ | 1 | $0<e<2047$ | $f$ | $-2^{e-1023}(1.f)$ |
| 非规格化正数 | 0 | 0 | $f\neq0$ | $2^{-126}(0.f)$ | 0 | 0 | $f\neq0$ | $2^{-1022}(0.f)$ |
| 非规格化负数 | 1 | 0 | $f\neq0$ | $-2^{-126}(0.f)$ | 1 | 0 | $f\neq0$ | $-2^{-1022}(0.f)$ |

# 3.5 浮点数据在机内的表示形式



规格化浮点数： $\pm 1.XXXX * 2^n$ B (不存贮整数位1)

n (移码) 范围： [00000001, 11111110]

因此，规格化单精度浮点指数范围：[-126, 127]

尾数范围：[000…000 (23个0), 111…111]

规格化单精度浮点数范围:

$\pm[1.0*2^{-126}, 1.111\ldots111*2^{127}]$ B ⇔

$\pm[1.175494351*10^{-38}, 3.402823466*10^{38}]$ D

Q：规格化浮点数能表示0和很小的数(如 $2^{-130}$ )吗？

# 3.5 浮点数据在机内的表示形式



最小的规格化正浮点数：$1.0*2^{-126}$

非规格化浮点数： $\pm 0.XXXX * 2^{-126}$ B

规定：阶码(移码) = 00000000 （尾数不为0）

非规格化单精度浮点数范围:

$\pm[0.000\ldots001*2^{-126}, 0.111\ldots111*2^{-126}]$ B ⇔

$\pm[1.4013*10^{-45}, 1.17548*10^{-38}]$ D

**对于区间$(-1.4013*10^{-45}, 0)$、$(0, 1.4013*10^{-45})$的数，float是无法表示的 !!!**

# 3.5 浮点数据在机内的表示形式



## 特殊数值

+∞ 符号位0、阶码全1、尾数为0

0-1111111 1-0000000 00000000 00000000

-∞ 符号位1、阶码全1、尾数为0

1-1111111 1-0000000 00000000 00000000

NaN (Not a Number) 符号位1或0、阶码全1、尾数不为0

例如：0-1111111 1-0000000 00000000 00000001

一些因素会引起 NaN（非数值），如 0 / 0、sqrt(-1) 等。

+0 符号位0、阶码全0、尾数为0

0-0000000 0-0000000 00000000 00000000

-0 符号位1、阶码全0、尾数为0

1-0000000 0-0000000 00000000 00000000

# 3.5 浮点数据在机内的表示形式



Problem1:

```
float  a = 10000;
float  b = a + 0.0001f;
bool  c1 = (a == b);
bool  c2 = (10000.0f < 10000.0001f);
```

c1= true
c2 = false

**c1, c2 = true or false ?**

Problem2:

```
float  a = 0x65536;
float  b;
```

a= 415030
48 ca a6 c0
b = 48 ca a6 c1
415030.031

**求满足 b > a 的最小 b。**

# “Father” of the IEEE 754 standard

80年代初，各个机器内部的浮点数表示格式还没有统一

相互不兼容，机器之间传送数据时，带来麻烦

1970年代后期，IEEE成立委员会着手制定浮点数标准

1985年完成浮点数标准IEEE 754的制定

**This standard was primarily the work of one person, UC Berkeley math professor William Kahan.**

www.cs.berkeley.edu/~wkahan/ieee754status/754story.html

**Prof. William Kahan**

# 3.5 浮点数据在机内的表示形式



Q：为什么指数使用移码表示，而不是补码表示？

在加法运算时，要对阶，用无符号数比较、移位更简单。

Q：为什么规格化时，用 $\pm 1.XXXX * 2^n$ B ，

而不是 $\pm 0.1XXXX * 2^n$ B ？

能够保存更多的有效位数，指数的表示范围也更大。

# 3.6 数据类型转换



同样一个数值，分别用 short, int, float, double 表示，结果相同吗？

int i; float f; double d;

Q: i == (int)(float)i; ?

```
#include <stdio.h>

int main()
{
    int  i;
    i = 1234567890;
    printf("%d \n", i);
    i = (int)(float)i;
    printf("%d \n", i);
    return 0;
}
```

Microsoft Visual Studio 调试控制台

```
1234567890
1234567936
```

# 3.6 数据类型转换



同样一个数值，分别用 short, int, float, double 表示，结果相同吗？

int i; float f; double d;

Q: i == (int)(double)i; ?

```
int main()
{
    int  i;
    i = 1234567890;
    printf("%d \n", i);
    i = (int)(float)i;
    printf("%d \n", i);
    i = 1234567890;
    printf("%d \n", i);
    i = (int)(double)i;
    printf("%d \n", i);
    return 0;
```

Microsoft Visual Studio 调试控制台

```
1234567890
1234567936
1234567890
1234567890
```

float 23 位尾数

double 52位尾数

# 3.6 数据类型转换



同样一个数值，分别用 short, int, float, double 表示，结果相同吗？

```
int  i;  float  f;  double  d;
```

Q: `f == (float)(int)f;` ?

# 结构数组的数据存放



通过实例展示结构数组的存放结果

不同编译开关下（对齐方式）结构数组数据的存放

# 总结



数据存储的基本形式

有符号整数和无符号整数的表示与存储

字符串的表示方法与存储

浮点数据的表示方法与存储

地址类型转换 与 数据类型转换